LLDB

```
#include <iostream>
#include "rabbit.h"
#include <plog/Log.h>
#include <plog/Initializers/RollingFileInitializer.h>

int main() {
  int x {};
  int y {};
  int* x_ptr = &x;
  int* y_ptr = &y;

  int h {};

  h = 10212;
  plog::init(plog::debug, "log.txt");

  moveup(x_ptr, y_ptr);

  PLOGD << "rabbit position "<< x << "," << y;

  moveup(x_ptr, y_ptr);

  PLOGD << "rabbit position "  << x << "," << y;

  movedown(x_ptr, y_ptr);

  PLOGD << "rabbit position " << x << "," << y;


  return 0;
}
```

Run debugger

```
% lldb main
```

Step

Step through each line or code (i.e. code expression)

(lldb) s Or (lldb) s

Jump

Jump to line of code within current stack frame

(lldb) jump 9 Or (lldb) j 9

Jump

Jump to instruction address

```
(lldb) j *0x0000000100000820
```

Note: * is required when jumping to a literal address value

General Purpose Registers

Read Registers

(lldb) register read

```
(lldb) register read
General Purpose Registers:
       rax = 0x000000010000b0a8  plog::Logger<0>& plog::init<0>(plog::Severity, plog::IAppender*)::logger
       rbx = 0x0000000100601bd0
       rcx = 0x0000600003358010
       rdx = 0x0000000000000003
       rdi = 0x0000000000000400
       rsi = 0x0000000000000020
       rbp = 0x00007ff7bfeff540
       rsp = 0x00007ff7bfefefd0
        r8 = 0x00000000000003ff
        r9 = 0x00000000819fe803
       r10 = 0x00000000000003fd
       r11 = 0x0000000000000002
       r12 = 0x00007ff7bfeff570
       r13 = 0x0000000000000000
       r14 = 0x0000000100000800  main`main at main.cpp:6
       r15 = 0x00007ff7bfeff6e0
       rip = 0x0000000100000812  main`main + 18 at main.cpp:7:7
    rflags = 0x0000000000000206
        cs = 0x000000000000002b
        fs = 0x0000000000000000
        gs = 0x0000000000000000
```

Frame

View current stack frame

(lldb) frame select

Print

Print variable

(lldb) print (x)

Print address of variable

(lldb) print &(x)

Memory

View memory

(lldb) x 0x00007ff7bfeff51c

Address of interest

e.g.

10212 = 0x27E4

```
(int *) 0x00007ff7bfeff51c
(lldb) print (h)
(int) 10212
(lldb) print &(h)
(int *) 0x00007ff7bfeff51c
(lldb) x 0x00007ff7bfeff51c
0x7ff7bfeff51c: e4 27 00 00 34 f5 ef bf f7 7f 00 00 38 f5 ef bf  .'..4.......8...
0x7ff7bfeff52c: f7 7f 00 00 70 f5 ef bf 01 00 00 00 00 00 00 00  ....p...........
```

Big Endian

Addess	0xFF51C	0xFF51D	...
Byte (Hex)	E4	27	...

Backtrace

View CallStack

(lldb) thread backtrace or (lldb) bt

e.g. main thread calls plog routine (callee)

```
(lldb) bt
* thread #1, queue = 'com.apple.main-thread', stop reason = step in
  * frame #0: 0x0000000100000ca6 main`plog::init(maxSeverity=debug, fileName="log.txt", maxFileSize=0, maxFiles=0) at RollingFileInitializer.h:50:47
    frame #1: 0x0000000100000855 main`main at main.cpp:15:3
    frame #2: 0x00007ff813cda345 dyld`start + 1909
```

e.g. Thread return back to main routine

```
(lldb) bt
* thread #1, queue = 'com.apple.main-thread', stop reason = step over
  * frame #0: 0x0000000100000855 main`main at main.cpp:17:10
    frame #1: 0x00007ff813cda345 dyld`start + 1909
```